Object-Oriented Design Pattern Laboratory Examination
Project 1: Agricultural Automation System (AAS)

Suggested duration: 3 hours
Total marks: 100%
Written examination: Phases 1–4 — 50% marks
Java implementation: Phase 5 — 50% marks

Project scenario

A farm currently relies on manual monitoring of soil moisture, irrigation, fertilizer application, and crop health. This causes delayed decisions, unnecessary water consumption, and difficulties responding to changing weather conditions.

Develop an **Agricultural Automation System (AAS)** that monitors farm conditions, supports irrigation decisions, and sends notifications to farmers. The system should support different sensor types, alternative irrigation strategies, and integration with an external weather service.

Consider the following actors: **Farmer, Farm Administrator, IoT Sensor, Weather Service, Drone System, and AI Analytics System**. Clearly state which features your examination solution implements and which remain outside its scope.

Instructions

- Complete **Phases 1–4 on paper** and **Phase 5 in Java**.
- Keep class names and responsibilities consistent across your written design and implementation.
- Use simulated sensor readings and weather data. Actual hardware, internet access, a database, and AI model training are not required.
- Implement a console application demonstrating the core irrigation workflow.
- State reasonable assumptions wherever requirements are unspecified.

Phase 1: Requirement Analysis — 10% marks

Question 1

Analyze the Agricultural Automation System and complete the following tasks:

a. Describe the system background and identify **three problems** with the existing manual farming process. **[2 marks]**

b. State **one primary objective** and **four specific objectives** of the proposed system. **[2 marks]**

c. Identify **four actors** and explain how each interacts with the system. Define the system boundary so that external devices and services are clearly distinguished from internal software components. **[2 marks]**

d. Specify **four functional requirements** and **two non-functional requirements**. At least two functional requirements must concern the irrigation workflow you will implement. **[4 marks]**

Phase 2: Object Identification — 10% marks

Question 2

Identify and classify suitable objects for the AAS.

a. Identify **five entity classes** and provide at least **two attributes** for each class. **[4 marks]**

b. Identify **three control classes** and explain their responsibilities. **[3 marks]**

c. Identify **three boundary classes** and explain the external interaction each supports. **[3 % marks]**

Present your answer using the following format:

Class name	Classification	Attributes or responsibilities
Your identified class	Entity / Control / Boundary	Relevant details

You may consider classes such as Crop, Field, SoilSensor, WeatherData, IrrigationPump, IrrigationController, FarmerMobileApp, and WeatherAPIInterface. Justify your classification rather than simply copying the examples.

Phase 3: CRC Card Development — 10 % marks

Question 3

Prepare **six CRC cards** for classes involved in monitoring soil conditions, obtaining weather information, making irrigation decisions, and notifying the farmer.

Each card must include:

- **Class name**
- **At least two meaningful responsibilities**
- **Collaborating classes**

Your cards must collectively include at least **one entity, one control, and one boundary class**. Ensure that the collaborators support the responsibilities assigned to each class.

Phase 4: UML Modeling — 20 % marks

Question 4

a. UML class diagram — 12 % marks

Draw a UML class diagram for the core irrigation subsystem containing **at least eight classes or interfaces**.

Include:

- Class and interface names.
- Important attributes and methods.
- Visibility indicators.
- Associations and relevant multiplicities.
- Interface realization or inheritance where appropriate.
- Classes supporting your selected design patterns.

Show aggregation or composition **only where the ownership and lifecycle relationship justifies it**.

b. UML sequence diagram — 8 % marks

Draw a sequence diagram for the following operation:

A farmer requests an irrigation check. The system reads soil moisture, obtains weather information through an adapter, applies the selected irrigation strategy, activates or leaves the pump off, and sends the farmer a notification.

Include:

- The farmer and relevant software objects.
- Messages in execution order.
- An alt fragment showing **irrigation required** and **irrigation not required**.
- A notification following the decision.

Phase 5: Java Implementation — 50 % marks

Question 5

Implement a console-based irrigation subsystem using **all five design patterns** below.

Pattern	Required implementation	% marks
Singleton	Implement FarmDataManager with a private constructor and a controlled access method. Store simulated farm information and demonstrate that two access calls return the same instance.	6
Factory	Implement SensorFactory to create at least two sensor types, such as SoilMoistureSensor and TemperatureSensor, through a common Sensor interface. Handle unsupported sensor types. A Simple Factory is acceptable; identify the variant used.	8
Observer	Implement an alert publisher and at least two subscribers, such as FarmerNotifier and FarmDashboard. Demonstrate subscription, notification, and unsubscription.	8
Strategy	Implement an IrrigationStrategy interface with NormalIrrigationStrategy and WaterSavingStrategy. Allow the controller to switch strategies at runtime.	8

Pattern	Required implementation	% marks
Adapter	Implement WeatherService and a WeatherAPIAdapter that wraps a simulated external weather class with an incompatible method or data format.	8
Integration and testing	Integrate the patterns into a runnable irrigation workflow and demonstrate the required tests below.	12
Total		**50**

Required irrigation rules

Use these **simplified examination rules**, rather than treating them as agricultural recommendations:

- **Normal strategy:** Irrigate when soil moisture is **below 40%** and rain is **not expected**.
- **Water-saving strategy:** Irrigate when soil moisture is **below 25%** and rain is **not expected**.
- Otherwise, leave the pump off.
- Reject soil-moisture readings outside **0–100%**.

The controller must use weather information obtained through the adapter and publish the irrigation decision to its registered observers.

Required test demonstrations

Test	Input or action	Expected behavior
1	Moisture 30%, no rain, normal strategy	Pump on
2	Moisture 30%, no rain, water-saving strategy	Pump off
3	Moisture 20%, rain expected, either strategy	Pump off
4	Moisture 40%, no rain, normal strategy	Pump off
5	Moisture 25%, no rain, water-saving strategy	Pump off
6	Moisture −5% or 105%	Invalid reading rejected; pump not activated
7	Publish an alert with two observers registered	Both observers receive the alert

Test	Input or action	Expected behavior
8	Unsubscribe one observer and publish another alert	Only the remaining observer receives it
9	Request FarmDataManager twice	Both references identify the same instance
10	Create both supported sensor types and request an unsupported type	Correct sensors returned; unsupported request handled clearly

Submission requirements

Submit the handwritten answers for Phases 1–4, Java source files for Phase 5, brief compilation and execution instructions, and a test-results table showing expected and actual outcomes.